

■ MongoDB Shell Command Guide

Beginner → Advanced · Every Command Explained
Why to Use · How to Use · Expected Output

Covers: mongosh · CRUD · Aggregation · Indexes · Auth · Transactions

Table of Contents

1. Getting Started & Connecting
2. Database Commands
3. Collection Commands
4. INSERT Commands
5. FIND / QUERY Commands
6. Query Operators
7. UPDATE Commands
8. DELETE Commands
9. Aggregation Pipeline
10. Index Commands
11. User & Authentication
12. Admin & Stats
13. Transactions
14. Quick Reference Cheat Sheet

1. Getting Started & Connecting

Connect to Local MongoDB

Why use it: Opens the MongoDB shell connected to your local MongoDB server running on default port 27017.

How to use: Just type `mongosh` in your terminal. Make sure MongoDB service is running first.

Command:

```
mongosh
```

Expected Output:

```
Current Mongosh Log ID: 64f1a2b3c4d5e6f7a8b9c0d1

Connecting to: mongodb://127.0.0.1:27017/

Using MongoDB: 7.0.0

test>
```

Connect to Remote MongoDB

Why use it: Connect to a MongoDB server on a different machine or cloud (like MongoDB Atlas).

How to use: Replace IP and port with your server details.

Command:

```
mongosh "mongodb://192.168.1.10:27017"

# For MongoDB Atlas:

mongosh "mongodb+srv://user:pass@cluster.mongodb.net/mydb"
```

Expected Output:

```
Connecting to: mongodb://192.168.1.10:27017/

test>
```

Connect with Username & Password

Why use it: Authenticate while connecting — needed when MongoDB has access control enabled.

Command:

```
mongosh --host localhost --port 27017 \

    --username admin --password secret \

    --authenticationDatabase admin
```

Expected Output:

```
Successfully authenticated as admin.

test>
```

Exit the Shell

Why use it: Cleanly close the MongoDB shell session.

Command:

```
exit  
  
# or  
  
quit()
```

Expected Output:

```
Goodbye!
```

2. Database Commands

List All Databases — `show dbs`

Why use it: See every database on the server along with its storage size.

How to use: Only databases with at least one document appear.

Command:

```
show dbs
```

Expected Output:

```
admin 40.00 KiB
config 72.00 KiB
local 72.00 KiB
myShop 512.00 KiB
```

Switch / Create Database — `use`

Why use it: Switch to a database. If it doesn't exist, MongoDB creates it when you first insert data.

How to use: You don't need to create a database first — just use it.

Command:

```
use myShop
```

Expected Output:

```
switched to db myShop

myShop>
```

Show Current Database — `db`

Why use it: Check which database you are currently working in.

Command:

```
db
```

Expected Output:

```
myShop
```

Drop (Delete) Current Database

Why use it: Permanently deletes the current database and ALL its data. Use with caution!

How to use: Switch to the database first with ``use dbName``, then run this command.

Command:

```
db.dropDatabase()
```

Expected Output:

```
{ ok: 1, dropped: 'myShop' }
```

3. Collection Commands

Show All Collections — `show collections`

Why use it: Lists all collections (like SQL tables) inside the current database.

Command:

```
show collections
```

Expected Output:

```
products
users
orders
```

Create a Collection

Why use it: Explicitly creates a collection. Normally MongoDB creates it automatically on first insert.

Command:

```
db.createCollection("products")
```

Expected Output:

```
{ ok: 1 }
```

Rename a Collection

Why use it: Rename an existing collection without moving or copying data.

Command:

```
db.products.renameCollection("items")
```

Expected Output:

```
{ ok: 1 }
```

Drop (Delete) a Collection

Why use it: Permanently removes a collection and ALL its documents.

Command:

```
db.products.drop()
```

Expected Output:

```
true
```

Get All Collection Names as Array

Why use it: Returns a JavaScript array of collection name strings — useful in scripts.

Command:

```
db.getCollectionNames()
```

Expected Output:

```
[ 'orders', 'products', 'users' ]
```

4. INSERT Commands

insertOne() — Insert a Single Document

Why use it: Adds one document (row) to a collection. MongoDB auto-generates a unique `_id`.

How to use: Use `insertOne()` when you have a single document to add.

Command:

```
db.users.insertOne({  
  
  name: "Alice",  
  
  age: 25,  
  
  email: "alice@example.com"  
  
})
```

Expected Output:

```
{  
  
  acknowledged: true,  
  
  insertedId: ObjectId('64f1a2b3c4d5e6f7a8b9c0d2')  
  
}
```

insertMany() — Insert Multiple Documents

Why use it: Adds multiple documents at once. Much faster than calling insertOne() repeatedly.

How to use: Pass an array of documents. Returns all inserted IDs.

Command:

```
db.users.insertMany([
  { name: "Bob", age: 30 },
  { name: "Charlie", age: 22 },
  { name: "Diana", age: 28 }
])
```

Expected Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('64f1...d3'),
    '1': ObjectId('64f1...d4'),
    '2': ObjectId('64f1...d5')
  }
}
```

5. FIND / QUERY Commands

find() — Get All Documents

Why use it: Returns every document in the collection. Like `SELECT *` in SQL.

Command:

```
db.users.find()
```

Expected Output:

```
[
  { _id: ObjectId('...'), name: 'Alice', age: 25 },
  { _id: ObjectId('...'), name: 'Bob', age: 30 },
  { _id: ObjectId('...'), name: 'Charlie', age: 22 }
]
```

find() with Filter

Why use it: Returns only documents that match your condition. Like `WHERE` in SQL.

How to use: Put the filter as the first argument — a JSON object.

Command:

```
db.users.find({ age: 25 })
```

Expected Output:

```
[ { _id: ObjectId('...'), name: 'Alice', age: 25 } ]
```

findOne() — Get First Match

Why use it: Returns only the first document that matches. Fast when you only need one result.

Command:

```
db.users.findOne({ name: "Alice" })
```

Expected Output:

```
{ _id: ObjectId('...'), name: 'Alice', age: 25, email: 'alice@example.com' }
```

Projection — Select Specific Fields

Why use it: Control which fields to show (1 = include, 0 = exclude). Like `SELECT col1, col2` in SQL.

How to use: Second argument is the projection. `_id` is included by default unless you set it to 0.

Command:

```
db.users.find({}, { name: 1, age: 1, _id: 0 })
```

Expected Output:

```
[ { name: 'Alice', age: 25 },
  { name: 'Bob', age: 30 },
  { name: 'Charlie', age: 22 } ]
```

sort(), limit(), skip() — Pagination

Why use it: Sort results, limit number returned, and skip documents — essential for pagination.

Command:

```
// Sort by age ascending (1) or descending (-1)

db.users.find().sort({ age: 1 })

// Get only first 2 results

db.users.find().limit(2)

// Page 2: skip 5, get next 5

db.users.find().skip(5).limit(5)
```

Expected Output:

```
// sort result:

[ { name: 'Charlie', age: 22 }, { name: 'Alice', age: 25 }, ... ]

// limit result:

[ { name: 'Alice', age: 25 }, { name: 'Bob', age: 30 } ]
```

countDocuments() — Count Results

Why use it: Count how many documents match a filter — fast and efficient.

Command:

```
db.users.countDocuments() // count all

db.users.countDocuments({ age: { $gt: 24 } }) // count filtered
```

Expected Output:

```
3 // total

2 // age > 24
```

6. Query Operators

Comparison Operators (\$gt, \$lt, \$gte, \$lte, \$ne, \$eq)

Why use it: Filter documents by comparing field values — greater than, less than, not equal, etc.

Command:

```
db.users.find({ age: { $gt: 24 } }) // age > 24

db.users.find({ age: { $gte: 25 } }) // age >= 25

db.users.find({ age: { $lt: 30 } }) // age < 30

db.users.find({ age: { $lte: 30 } }) // age <= 30

db.users.find({ age: { $ne: 25 } }) // age != 25

db.users.find({ age: { $eq: 25 } }) // age == 25
```

Expected Output:

```
// $gt: 24 returns age 25 and 30

[ { name: 'Alice', age: 25 }, { name: 'Bob', age: 30 } ]
```

\$in and \$nin — Match from a List

Why use it: Check if a field value is (or isn't) inside a given array of values.

Command:

```
db.users.find({ age: { $in: [25, 30] } }) // age is 25 OR 30

db.users.find({ age: { $nin: [25, 30] } }) // age is NOT 25 or 30
```

Expected Output:

```
// $in result:

[ { name: 'Alice', age: 25 }, { name: 'Bob', age: 30 } ]

// $nin result:

[ { name: 'Charlie', age: 22 } ]
```

\$and, \$or, \$not, \$nor — Logical Operators

Why use it: Combine multiple conditions — like AND / OR / NOT in SQL.

Command:

```
// AND: age > 20 AND name is Alice

db.users.find({ $and: [{ age: { $gt: 20 } }, { name: "Alice" }] })

// OR: age is 25 OR name is Bob

db.users.find({ $or: [{ age: 25 }, { name: "Bob" }] })

// NOT: age is NOT greater than 25

db.users.find({ age: { $not: { $gt: 25 } } })
```

Expected Output:

```
// $and result: [ { name: 'Alice', age: 25 } ]

// $or result: [ { name: 'Alice', age: 25 }, { name: 'Bob', age: 30 } ]

// $not result: [ { name: 'Alice', age: 25 }, { name: 'Charlie', age: 22 } ]
```

Regex Search — Pattern Matching

Why use it: Find documents where a string field matches a pattern. Like LIKE in SQL.

Command:

```
db.users.find({ name: /^A/ }) // starts with A

db.users.find({ name: /ice$/i }) // ends with "ice", case-insensitive

db.users.find({ name: { $regex: "ali", $options: "i" } })
```

Expected Output:

```
[ { name: "Alice", age: 25 } ]
```

7. UPDATE Commands

updateOne() — Update First Matching Document

Why use it: Updates only the first document that matches the filter. Safer than updateMany.

How to use: First arg = filter (who to update). Second arg = what to change.

Command:

```
db.users.updateOne(
  { name: "Alice" }, // filter: find Alice
  { $set: { age: 26 } } // update: set age to 26
)
```

Expected Output:

```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

updateMany() — Update All Matching Documents

Why use it: Updates ALL documents that match the filter. Use carefully!

Command:

```
db.users.updateMany(
  { age: { $lt: 25 } },
  { $set: { category: "junior" } }
)
```

Expected Output:

```
{
  acknowledged: true,
  matchedCount: 2,
  modifiedCount: 2
}
```

Update Operators (\$set, \$unset, \$inc, \$push, \$pull)

Why use it: Special operators that control HOW a field is updated.

Command:

```
// $set – set a field value

db.users.updateOne({ name:"Bob" }, { $set: { city: "Dhaka" } })


// $unset – remove a field entirely

db.users.updateOne({ name:"Bob" }, { $unset: { city: "" } })


// $inc – increment a number

db.users.updateOne({ name:"Bob" }, { $inc: { age: 1 } })


// $push – add item to array

db.users.updateOne({ name:"Bob" }, { $push: { tags: "vip" } })


// $pull – remove item from array

db.users.updateOne({ name:"Bob" }, { $pull: { tags: "vip" } })
```

Expected Output:

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

Upsert — Update or Insert

Why use it: If a matching document is found, update it. If not found, insert a new one.

Command:

```
db.users.updateOne(
  { name: "Eve" },
  { $set: { age: 29 } },
  { upsert: true } // insert if not found
)
```

Expected Output:

```
{
  acknowledged: true,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedId: ObjectId('64f1...d9') // new document created
}
```

replaceOne() — Replace Entire Document

Why use it: Replaces the entire document (except `_id`). Different from `$set` which only updates specific fields.

Command:

```
db.users.replaceOne(
  { name: "Alice" },
  { name: "Alice", age: 27, city: "Dhaka", role: "admin" }
)
```

Expected Output:

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

8. DELETE Commands

deleteOne() — Delete First Match

Why use it: Deletes only the first document that matches the filter.

How to use: Always filter precisely — if multiple docs match, only the first is deleted.

Command:

```
db.users.deleteOne({ name: "Charlie" })
```

Expected Output:

```
{ acknowledged: true, deletedCount: 1 }
```

deleteMany() — Delete All Matches

Why use it: Deletes ALL documents that match the filter.

Command:

```
db.users.deleteMany({ age: { $lt: 20 } }) // delete underage users

db.users.deleteMany({}) // delete EVERYTHING (careful!)
```

Expected Output:

```
{ acknowledged: true, deletedCount: 3 }
```

findOneAndDelete() — Find, Return, and Delete

Why use it: Finds a document, returns it to you, and then deletes it — atomically.

How to use: Useful when you need to process a document and remove it at the same time.

Command:

```
db.users.findOneAndDelete({ name: "Bob" })
```

Expected Output:

```
{ _id: ObjectId('...'), name: 'Bob', age: 30 } // returned before deletion
```

9. Aggregation Pipeline

Aggregation processes documents through a pipeline of stages. Each stage transforms the data before passing it to the next stage.

\$match — Filter Documents (like find)

Why use it: Filters documents passing through the pipeline. Always put \$match early to improve performance.

Command:

```
db.users.aggregate([
  { $match: { age: { $gte: 25 } } }
])
```

Expected Output:

```
[ { name: 'Alice', age: 25 }, { name: 'Bob', age: 30 } ]
```

\$group — Group and Aggregate

Why use it: Groups documents by a field and computes statistics like count, sum, average.

How to use: Use `_id`: null to aggregate across ALL documents.

Command:

```
db.orders.aggregate([
  { $group: {
    _id: "$status", // group by status field

    total: { $sum: "$amount" }, // sum of amounts per group

    count: { $sum: 1 } // count per group
  }}
])
```

Expected Output:

```
[ { _id: "paid", total: 1500, count: 5 },
  { _id: "pending", total: 800, count: 3 } ]
```

\$sort, \$limit, \$skip

Why use it: Sort, limit, and paginate aggregation results — same concept as chaining on find().

Command:

```
db.users.aggregate([  
  
  { $sort: { age: -1 } }, // sort by age descending  
  
  { $skip: 0 }, // skip 0 docs  
  
  { $limit: 3 } // return top 3  
  
])
```

Expected Output:

```
[ { name: 'Bob', age: 30 }, { name: 'Alice', age: 25 }, { name: 'Charlie', age: 22 } ]
```

\$project — Reshape / Select Fields

Why use it: Include, exclude, or rename fields in the output. Can also create computed fields.

Command:

```
db.users.aggregate([  
  
  { $project: {  
  
    _id: 0,  
  
    fullName: "$name", // rename field  
  
    age: 1,  
  
    isAdult: { $gte: ["$age", 18] } // computed boolean  
  
  }}  
  
])
```

Expected Output:

```
[ { fullName: "Alice", age: 25, isAdult: true },  
  
  { fullName: "Bob", age: 30, isAdult: true },  
  
  { fullName: "Charlie", age: 22, isAdult: true } ]
```

\$lookup — Join Two Collections

Why use it: Joins documents from another collection — like SQL LEFT JOIN.

Command:

```
db.orders.aggregate([
  { $lookup: {
    from: "users", // collection to join
    localField: "userId", // field in orders
    foreignField: "_id", // field in users
    as: "userInfo" // output array field
  }}
])
```

Expected Output:

```
[ {
  _id: ObjectId("..."), amount: 300,
  userInfo: [{ _id: ObjectId("..."), name: "Alice" }]
} ]
```

\$unwind — Flatten Arrays

Why use it: Deconstructs an array field — creates one document per array element.

How to use: Useful before \$group to count array elements.

Command:

```
// user has: { name: "Alice", tags: ["vip", "new"] }

db.users.aggregate([
  { $unwind: "$tags" }
])
```

Expected Output:

```
[ { name: "Alice", tags: "vip" },
  { name: "Alice", tags: "new" } ]
```

Full Pipeline Example

Why use it: A real-world example combining multiple stages to get top cities by average age.

Command:

```
db.users.aggregate([

  { $match: { age: { $gt: 18 } } }, // only adults

  { $group: { _id: "$city",

              avgAge: { $avg: "$age" },

              count: { $sum: 1 } } }, // group by city

  { $sort: { avgAge: -1 } }, // highest avg first

  { $limit: 5 }, // top 5 cities

  { $project: { _id: 0, city: "$_id",

                avgAge: 1, count: 1 } } // clean output

])
```

Expected Output:

```
[ { city: "Dhaka", avgAge: 29.5, count: 12 },
  { city: "Khulna", avgAge: 27.0, count: 8 } ]
```

10. Index Commands

Indexes speed up queries dramatically by avoiding full collection scans. Think of them like a book's index — without it MongoDB must read every page.

createIndex() — Create a Single Field Index

Why use it: Speeds up queries that filter or sort on the indexed field.

Command:

```
db.users.createIndex({ name: 1 }) // 1 = ascending

db.users.createIndex({ age: -1 }) // -1 = descending
```

Expected Output:

```
{ numIndexesBefore: 1, numIndexesAfter: 2, ok: 1 }
```

Unique Index

Why use it: Ensures no two documents can have the same value for this field — like UNIQUE constraint in SQL.

How to use: Attempting to insert a duplicate email will throw an error.

Command:

```
db.users.createIndex({ email: 1 }, { unique: true })
```

Expected Output:

```
{ numIndexesBefore: 2, numIndexesAfter: 3, ok: 1 }
```

Compound Index

Why use it: Index on multiple fields. Useful when you frequently query/sort by two fields together.

Command:

```
db.users.createIndex({ name: 1, age: -1 })
```

Expected Output:

```
{ numIndexesBefore: 3, numIndexesAfter: 4, ok: 1 }
```

Text Index — Full-Text Search

Why use it: Enables text search across string fields using \$text operator.

Command:

```
db.products.createIndex({ description: "text" })

// Then search:

db.products.find({ $text: { $search: "laptop wireless" } })
```

Expected Output:

```
[ { _id: ObjectId("..."), name: "Dell Laptop", ... } ]
```

TTL Index — Auto-Expire Documents

Why use it: Automatically deletes documents after a set number of seconds. Great for sessions/logs.

How to use: The field must be a Date type. MongoDB runs a background task every 60 seconds.

Command:

```
db.sessions.createIndex(  
  
  { createdAt: 1 },  
  
  { expireAfterSeconds: 3600 } // delete after 1 hour  
  
)
```

Expected Output:

```
{ numIndexesBefore: 1, numIndexesAfter: 2, ok: 1 }
```

getIndexes() — List All Indexes

Why use it: See all existing indexes on a collection.

Command:

```
db.users.getIndexes()
```

Expected Output:

```
[  
  
  { key: { _id: 1 }, name: "_id_" },  
  
  { key: { name: 1 }, name: "name_1" },  
  
  { key: { email: 1 }, name: "email_1", unique: true }  
  
]
```

dropIndex() / dropIndexes()

Why use it: Remove a specific index or all indexes from a collection.

Command:

```
db.users.dropIndex("name_1") // drop by index name  
  
db.users.dropIndexes() // drop ALL (except _id)
```

Expected Output:

```
{ nIndexesWas: 3, ok: 1 }
```

explain() — Analyze Query Performance

Why use it: Shows how MongoDB executes a query — whether it uses an index or does a full scan.

How to use: Look for IXSCAN (index scan) vs COLLSCAN (collection scan = slow).

Command:

```
db.users.explain("executionStats")

    .find({ name: "Alice" })
```

Expected Output:

```
executionStats: {
  nReturned: 1,
  totalDocsExamined: 1, // GOOD: only 1 doc scanned (index used)
  executionTimeMillis: 0
}

// Without index: totalDocsExamined = totalDocs (slow)
```

11. User & Authentication

createUser() — Create a New User

Why use it: Create a database user with specific roles and permissions.

How to use: Common roles: read, readWrite, dbAdmin, dbOwner, clusterAdmin, root

Command:

```
db.createUser({
  user: "shopAdmin",
  pwd: "securePass123",
  roles: [
    { role: "readWrite", db: "myShop" },
    { role: "read", db: "reporting" }
  ]
})
```

Expected Output:

```
{ ok: 1 }
```

getUsers() — List All Users

Why use it: See all users in the current database.

Command:

```
db.getUsers()
```

Expected Output:

```
{ users: [ { _id: "myShop.shopAdmin", user: "shopAdmin",
  roles: [{ role: "readWrite", db: "myShop" }] } ] }
```

changeUserPassword()

Why use it: Update the password for an existing user.

Command:

```
db.changeUserPassword("shopAdmin", "newStrongPass456")
```

Expected Output:

```
{ ok: 1 }
```

dropUser() — Delete a User

Why use it: Removes a user from the current database.

Command:

```
db.dropUser("shopAdmin")
```

Expected Output:

```
{ ok: 1 }
```

12. Admin & Stats

db.stats() — Database Statistics

Why use it: Shows database size, number of collections, documents, and storage usage.

Command:

```
db.stats()
```

Expected Output:

```
{
  db: "myShop",
  collections: 3,
  objects: 1500, // total documents
  avgObjSize: 128, // bytes per document
  dataSize: 192000, // total data in bytes
  storageSize: 262144
}
```

db.serverStatus() — Server Information

Why use it: Comprehensive server metrics: uptime, connections, memory, operations.

Command:

```
db.serverStatus()
```

Expected Output:

```
{
  host: "DESKTOP-ABC123",
  version: "7.0.0",
  uptime: 86400, // seconds
  connections: { current: 5, available: 838855 },
  mem: { resident: 128, virtual: 1024 }
}
```

db.currentOp() — Running Operations

Why use it: Shows all currently running database operations. Useful for finding slow queries.

Command:

```
db.currentOp()
```

Expected Output:

```
{
  inprog: [
    { opid: 12345, op: "query", ns: "myShop.users",
      secs_running: 5, query: { age: { $gt: 20 } } }
  ]
}
```

db.killOp() — Kill a Running Operation

Why use it: Stops a long-running or stuck database operation using its operation ID.

Command:

```
db.killOp(12345) // use opid from currentOp()
```

Expected Output:

```
{ info: "attempting to kill op", ok: 1 }
```

db.collection.stats() — Collection Statistics

Why use it: Detailed stats for a single collection including size, index info, and document count.

Command:

```
db.users.stats()
```

Expected Output:

```
{
  ns: "myShop.users",
  count: 150,
  size: 19200,
  avgObjSize: 128,
  nindexes: 3,
  totalIndexSize: 49152
}
```

13. Transactions

Transactions ensure multiple operations either ALL succeed or ALL fail — like a bank transfer. Requires MongoDB 4.0+ and a Replica Set or Sharded Cluster.

Multi-Document Transaction

Why use it: Perform multiple writes atomically — if any step fails, all changes are rolled back.

Command:

```
const session = db.getMongo().startSession()

session.startTransaction()

try {

  const db = session.getDatabase("myShop")

  // Deduct from account A

  db.accounts.updateOne(

    { user: "Alice" },

    { $inc: { balance: -100 } }

  )

  // Add to account B

  db.accounts.updateOne(

    { user: "Bob" },

    { $inc: { balance: +100 } }

  )

  session.commitTransaction() // COMMIT if all OK

  print("Transaction committed!")

} catch(err) {

  session.abortTransaction() // ROLLBACK on error

  print("Transaction aborted:", err)

} finally {

  session.endSession()

}
```

Expected Output:

```
Transaction committed!

// Or on error:

Transaction aborted: MongoServerError: ...
```

14. Quick Reference Cheat Sheet

Command	Purpose
<code>show dbs</code>	List all databases
<code>use <dbName></code>	Switch to a database
<code>db.dropDatabase()</code>	Delete current database
<code>show collections</code>	List all collections
<code>db.col.insertOne({...})</code>	Insert one document
<code>db.col.insertMany([...])</code>	Insert many documents
<code>db.col.find()</code>	Get all documents
<code>db.col.find({field:val})</code>	Filter documents
<code>db.col.findOne({...})</code>	Get first match
<code>db.col.find().sort({f:1})</code>	Sort results
<code>db.col.find().limit(n)</code>	Limit results
<code>db.col.countDocuments({})</code>	Count documents
<code>db.col.updateOne({}, {\$set})</code>	Update one document
<code>db.col.updateMany({}, {...})</code>	Update many documents
<code>db.col.deleteOne({...})</code>	Delete one document
<code>db.col.deleteMany({...})</code>	Delete many documents
<code>db.col.aggregate([...])</code>	Run aggregation pipeline
<code>db.col.createIndex({f:1})</code>	Create an index
<code>db.col.getIndexes()</code>	List indexes
<code>db.col.explain().find()</code>	Analyze query plan
<code>db.createUser({...})</code>	Create a user
<code>db.stats()</code>	Database statistics
<code>db.serverStatus()</code>	Server statistics
<code>db.currentOp()</code>	Running operations
<code>db.killOp(id)</code>	Kill an operation

■ Tip: Always test destructive commands (delete, drop, updateMany) on a backup first. Use explain() to check query performance. Add indexes on fields you query most often.